1 Lab 1

1. Identify system's HW:

```
# lspci && lsusb
```

These commands allow identifying the devices directly connected to PCI and USB buses.

2. Map the found hardware parts with the actual devices, plus try to find the devices not found using the previous commands. Free command to find RAM memory usage and total (useful for SWAP partition).

```
# free
# dmesg
```

3. Check if there are already made partitions, and if the sectors are contiguous. See spacing and filesystem:

```
# fdisk -f
# lsblk
```

Particiones

1. Find which is the device corresponding to the usb drive, it is in the form: /dev/usb

To avoid issues, unmount the USB device.

```
# umount /dev/usb
```

2. Partition the disk using gdisk.

Determine the geometry and the size of your disk. Create the partitions indicated in Table 1. Write the contents of the partition table to disk (remember to do so before leaving from the `gdisk` command).

Device	Code	Size	Mount-point	Comments
/dev/usb1	EFI System Partition (`EF00`)	512 MB	/boot/efi	This partition will hold the necessary EFI Boot information. It **MUST** be formatted with FAT32 (`vfat`) filesystem.
/dev/usb2	Linux (`8304`)	30 GB+	/	This partition will hold the main system. A typical Debian install requires around 5 GB, but it can grow—use at least 30 GB.
/dev/usb3	Linux (`8300`)	5 GB	/usr/local	We don't use much space here during the course; 5 GB should be enough. In a production system size according to your self-compiled apps.
/dev/usb4	Linux (`8302`)	100 GB+	/home	In general this is where most space should be devoted. You can put 100 GB or more.
/dev/usb5	Swap (`8200`)	2×RAM	—	Put twice the size of the machine's RAM.
/dev/usb6	Linux (`8300`)	20 GB	—	Reserved for future use; you don't need to create it now.

Taula 1: Partition layout for USB installation media

Creating the file systems

Initialize the file system on those partitions and prepare the swap area for use.

Initialize the swap:

```
# mkswap device
```

Create the filesystems:

```
# mkfs -t fstype device
```

Mounting the file systems

1. Mount the filesystems in a temporary directory to be able to install the software.

```
# mkdir /linux
```

2. Mount all filesystems inside such a mount point (/linux) ***(It is important to create the directories after mounting /linux but before mounting the rest)***, creating the appropriate directories inside, at the same time. Always inside /linux; for instance, / partition into /linux, /home partition into /linux/home...

```
# mount part dir
```

Installation of the base system

In our case, we will install from a prepackaged system image that is in the ASO FTP server: asoserver.pc.ac.upc.edu. You will have to decompress it into the USB disk. Cd into the filesystem that will become the root (/) of your new installation (remember, mounted on /linux).

```
# cd /linux
# sftp aso@asoserver.pc.ac.upc.edu
sftp> get <filename>
# tar xzf aso-install.tar.gz
# rm aso-install.tar.gz
```

*You can use `lcd` (change directory on local machine) and `cd` (change directory on sftp server)

Mounting auxiliary systems

We have to bind-mount the directories /dev, /sys, and /proc inside /linux, to temporarily expose the current existing devices to the new system.

```
# for i in /dev /dev/pts /proc /sys /run; do
> mount -B $i /linux/$i
> done
```

*bind-mounting: A bind mount is an alternate view of a directory tree. Classically, mounting creates a view of a storage device as a directory tree. A bind mount instead takes an existing directory tree and replicates it under a different point. The directories and files in the bind mount are the same as the original. Any modification on one side is immediately reflected on the other side, since the two views show the same data.

Use the `mount` command without any parameters to see which filesystems are mounted, and verify that all USB disk partitions are mounted correctly in the appropriate directories, including the `/dev` directory with the system device files.

System configuration

Before rebooting, you should configure the system mount points through the `/etc/fstab` file and install a boot loader. Configuration files in operating systems based on Unix/Linux are in the default directory `/etc` and almost always in text format. ***Configuring the*** `/etc/fstab` ***file***

Add your swap partition: `device none swap defaults 0 0`

Add root partition: `device none swap defaults 0 0`

Add the rest of filesystems you created previously: `device none swap defaults 0 0`

Leave the entry `/proc` and `/sys` without change, because they are pseudo-filesystems provided by the kernel (they're virtual).

Changing the root directory

You can change the root directory of your system, and temporarily use the software that you installed in the system, instead of the one currently available on the system.

```
# chroot /linux
```

Configuring the keyboard

```
# dpkg-reconfigure locales
# dpkg-reconfigure console-data
# dpkg-reconfigure keyboard-configuration
```

Configuring the boot process

Currently, we have the system installed, but we need to somehow point where our OS is to the BIOS so that next time you boot the computer it is done properly. To this end, we will install the GRUB boot manager.

```
# grub-install --target=x86 64-efi /dev/usb
```

This script prepares the `/boot` directory in order to contain the information needed to boot the machine. The steps it performs:

- Creates the directory `/boot/grub`

- Copy the files needed for GRUB to `/boot`

- Installs the bootloader in the MBR of the USB disk.

The system must inform GRUB which kernel must be used to properly boot. This can be accomplished by editing the file `/boot/grub/grub.cfg`. Debian comes with system tools that allow the automatic creation of such file in order to automatically update the `/boot/grub/grub.cfg` file to contain the UUID of the particular partition you are using. Run the script:

```
# update-grub
```

*get UUIDs with `blkid` (**note this is just a comment, not necessary for the process**).

Setting up the passwords

To change the password we need to update the file `/etc/shadow`. As expected the file has hashed passwords rather than plain text. In order to be able to change the passwords we can use the command `passwd [LOGIN]`. `[LOGIN]` is optional, if we're not using the root user we will not be able to use it, if we are, we can change other user's password (but never see their password). We should use passwd only if we are not a privileged user, this way we will change OUR password.

You can now exit the `chroot` shell

Unmount all filesystems and reboot using the `shutdown -r now` command.

Post-configuration

Boot the newly installed system. Remember, if you are on the Lab when the HP screen appears press F12 to pop up the Boot menu from the BIOS. It is important to select the Toshiba hard drive from the UEFI boot option. Don't select the Legacy boot option, as it will try to boot with the former MBR mechanism which we didn't configure. Login as `aso` using the password you set previously. When you need to have user privileges (i.e. become root), use the command `su`:

```
$ su
# privileged-command ← this will run privileged-command as root
# exit
```

Configuring filesystems

In file systems ext3 and ext4 we find certain properties that can be changed after formatting, with the tune2fs command. Using this command, change the frequency of checks of the filesystem in the usb2 partition to 28 days.

```
# tune2fs /dev/sdb1 -i 28d
```

Setting up network

First, we will do the network configuration by hand and then we will use DHCP to configure it permanently. Before starting, and to avoid mistakes we will flush the current network status running:

```
# ip link show
# ip link set dev <thernetiface> down
```

Manual configuration

1. Configure the network interface using `ip address` command.

2. Configure the routing table using `ip route` command.

3. Setup name resolution in `/etc/resolv.conf`

- Bring interface up:

```
# ip link set dev <eth iface> up
```

- Add default getway to routing table

```
# ip route add default via <gateway> dev <eth iface>
```

Permanent configuration

We want the network to be configured at bootime. First, delete the manual IP address running:

```
# ip address del 10.10.41.???/24 dev <ethernet IF>
```

Debian saves several network config files on `/etc/network`. Now we will use systemctl list-units so we can see all available services and status. You can see the networking.service. In the directory `/etc/network` there is a file `interfaces` that is where you configure different interfaces. Right now there is only configured the loopback interface. Add an entry in the `/etc/network/interfaces` file to configure your network interface. Follow these commands;

```
# auto <ethernet IF>
# iface <ethernet IF> inet static
# address 10.10.41.???
# network 10.10.41.0
# netmask 255.255.255.0
# gateway 10.10.41.1
```

Now, to check that we have configured it correctly, we could do a reboot. But in fact we do not need that. We can use the commands:

```
# ifup <ethernet IF>
# ifdown <ethernet IF>
```

Some final configuration

We need to install the command `sudo` now that we have internet. Do this:

```
# apt update
# apt install sudo
# usermod -a -G sudo aso
```

For this change to take effect, you have to close the session and log back in.

2 Lab 3

Shebang y permisos

- Primera línea (shebang):

  ```
  #!/bin/bash
  ```

  Indica al sistema qué intérprete usar (a menudo `/bin/bash`).

- Permisos de ejecución:

  ```
  chmod +x mi_script.sh
  ```

Sintaxis básica

Comentarios: `# Esto es un comentario`

Variables:

```
NOMBRE="Juan"
echo "Hola, $NOMBRE"
```

Lectura de entrada: `read -p "Tu nombre: " NOMBRE`

Estructuras de control

Condicional `if`

```
if [ -d "/ruta" ]; then
  echo "Existe"
else
  echo "No existe"
fi
```

Bucle `for`

```
for f in *.txt; do
  echo "Archivo: $f"
done
```

Bucle `while`

```
i=1
while [ $i -le 5 ]; do
  echo "Paso $i"
  ((i++))
done
```

Funciones

```
mi_funcion() {
  echo "Dentro de la función"
}
mi_funcion
```

Programas y utilidades esenciales

Para scripts de unas 10–20 líneas, conviene dominar:

- `echo`, `read`, `test` (`[ ]`)
- `grep`, `cut`, `awk`, `sed`
- `find`, `xargs`
- `sort`, `head`, `tail`
- `pushd`/`popd`, `cd`, `mkdir`, `rm`

Buenas prácticas

- Validar parámetros (`if [ $# -lt 1 ] ; then ... fi`).
- Usar `set -euo pipefail` al inicio para detectar errores.
- Documentar con comentarios claros.
- Probar paso a paso y manejar errores de comandos externos.

3 Lab 4

Bases de dades del sistema

Els fitxers principals per a la gestió d'usuaris al sistema són:

`/etc/passwd`**:** Base de dades d'usuaris. Totes les dades (excepte el password). Estructura:

```
username:password:UID:GID:UserInfo:HomeDir:Shell
```

`/etc/group`**:** Estructura:

```
groupname:password:GID:GroupUserList
```

Separats per comes. El camp `password` normalment no s'utilitza.

`/etc/shadow`**:** Conté la contrasenya encriptada. Estructura:

```
username:passwd:lastchanged:min:max:warn:inactive:expire
```

- **min:** dies mínims entre canvis de contrasenya.
- **max:** dies màxims de validesa de la contrasenya; passat això, es força el canvi.
- **warn:** dies abans que avisi de caducitat.
- **inactive:** dies després de la caducitat per desactivar el compte.
- **expire:** data (en segons des de l'1 de gen. 1970) en què el compte es deshabilita.

`/etc/aliases`**:** Aliases de correu.

Comandes úniques Per gestionar usuaris i grups:

`useradd | userdel | passwd | newusers | vipw | groupadd | groupdel | newgrp | vigr | chmod | chown | chgrp`

Els comptes d'usuari permeten al sistema diferenciar dades i processos, i protegir la informació de cada usuari. El kernel identifica els usuaris pel seu *username* i UID.

Creació d'un nou usuari Assignació d'un UID i modificació de la base de dades per:

- Paràmetres propis de l'usuari.
- Associar-lo a un grup.
- Copiar fitxers de configuració i personalitzar el `home`.

Afegir usuari a sudoers

1. Editar `/etc/sudoers` amb `visudo` o directament:

   ```
   [%grup / user]  ALL=(TARGET_USERS) [NOPASSWD:] ALLOWED_COMMANDS
   # Ex.: usuari ALL=(ALL) ALL
   ```

   Això dóna permís per executar qualsevol comanda com a qualsevol usuari.

2. O mitjançant:

   ```
   sudo usermod -aG sudo <username>
   ```

   això afegeix l'usuari al grup `sudo`.

Profile i Entorn d'Usuari

Fitxers de configuració global i local

- `/etc/profile`: entorn comú per a tots els usuaris, defineix variables com `PATH`.

- `~/.bash_profile`: entorn personal per a cada usuari (PATH, prompt, etc.).

Quan es crea un directori `home`, es copien els fitxers de `/etc/skel` perquè l'usuari tingui un entorn inicial.

Modificar el PATH Per assegurar que tots els usuaris tinguin `/usr/local/bin` i un directori `$HOME/bin`:

```
# afegit a ~/.bash_profile o /etc/profile
PATH=$PATH:$HOME/bin
export PATH
```

Personalitzar el prompt La variable d'entorn `PS1` permet canviar el prompt. Per exemple:

```
PS1="\u@\h:\w\$ "
```

on `_u` = usuari, `_h` = hostname, `_w` = directori de treball.

Creació Manual d'Usuaris

1. **Selecció de paràmetres:**
   - UID
   - Username
   - Home dir
   - Shell
   - Groups
2. **Editar la base de dades d'usuaris**
   Utilitzem `vipw` per bloquejar `/etc/passwd` i editar-lo com a root. Ex.:

   ```
   test:x:1001:1001:Test user aso:/home/test:/usr/bin/zsh
   ```

3. **Editar la base de dades de grups**
   De la mateixa manera amb `vigr`.

   ```
   test:x:1001:test
   ```

4. **Desactivar el compte de l'usuari**
   Canvia el shell a `/sbin/nologin` a `/etc/passwd`, o invalida el password amb:

   ```
   passwd -l <username>
   ```

5. **Crear el directori `home` i definir permisos**

   ```
   mkdir /home/usuari
   cp -a /etc/skel /home/usuari
   chmod 700 -R /home/usuari
   chown test:test /home/usuari
   ```

6. **Assignar un password**
   Modificar `/etc/shadow` com a superusuari o simplement:

   ```
   passwd <username>
   ```

   Altres eines per editar `/etc/shadow`: `chage`, `pwck`, `chfn`, `chsh`.

Creació Automàtica d'Usuaris

Tot el procés anterior es pot simplificar amb `useradd` i `adduser`. Per defecte els paràmetres es llegeixen de `/etc/default/useradd`. Ex.:

```
useradd \
  -d <Home_Dir> \
  -c <UserInfo> \
  -e <ExpireDate> \
  -g <GID> \
  -p <password> \
```

```
  -s <Shell> \
  -u <UID> \
  <username>
```

Per a diferents nivells d'accés sense canviar permisos dels `home`, és millor modificar els grups i els permisos compartits.

Eliminació i desactivació d'usuaris

Per donar de baixa un usuari s'han d'eliminar tots els seus fitxers, bústies, impressions, `cron`, i altres referències. També cal esborrar les línies associades als fitxers `/etc/passwd` i `/etc/group`. És habitual fer un backup dels fitxers fora del `home` abans de l'esborrat.

Eliminar fitxers de l'usuari

Listing 1: Eliminar tots els fitxers propietat de l'usuari

```
find / -user <user> -exec rm -fR {} \;
```

Backup amb `xargs`

Listing 2: Arxivar tots els fitxers de l'usuari

```
find / -user <user> | xargs tar -czf backup_<user>.tar.gz
```

Si els noms contenen espais, afegiu el flag `-d '\n'` a `xargs` perquè el delimitador sigui newline en lloc d'espai.

Desactivar abans de suprimir És recomanable desactivar el compte primer. Es pot:

- Canviar el shell a `/usr/sbin/nologin` amb `chsh`.
- Bloquejar el password amb `passwd -l`.

Missatge de shell personalitzat Poseu un petit script a `/usr/local/lib/no-login.sh`, per exemple:

Listing 3: Script de login desactivat

```
#!/usr/bin/tail -n 2
This account has been closed due to a security problem. Please
contact the system administrator.
```

A continuació:

```
chsh -s /usr/local/lib/no-login.sh <user>
```

Exemple de script complet

Listing 4: Backup i eliminació d'usuari

```
#!/bin/bash

usage="Usage: backdel.sh [username]"
if [ $# -eq 0 ]; then
  echo $usage; exit 1
fi

user=$1
archive="backup${user}.tar.gz"

# Backup: -T llegeix fitxers des de stdin
find / -user $user | tar -czf /home/copies/$archive -T -
# Esborrar fitxers
find / -user $user -exec rm -fR {} \;
# Canviar shell perque no pugui iniciar sessio
chsh -s /usr/local/lib/no-login.sh $user
```

Bloqueig i desbloqueig de `root`

```
# Bloquejar root
passwd -l root

# Desbloquejar root
sudo passwd root
```

setuid Permet que un executable s'executi amb els privilegis del seu propietari en comptes dels de l'usuari que el llança.

setgid Permet que un executable s'executi amb els privilegis del grup propietari, o fa que els fitxers creats en un directori heretin el grup del directori.

sticky bit Impedeix que usuaris no propietaris esborrin o renominin fitxers en un directori compartit.

4 Lab 5

Hard link: Los enlaces duros, asocian dos o más ficheros compartiendo el mismo inodo. Esto hace que cada enlace duro sea una copia exacta del resto de ficheros asociados, tanto de datos como de permisos, propietario. Al modificar los datos apuntados por cualquiera de ellos, se cambian los datos reales almacenados en disco, quedando modificados para todos por igual. El proceso de eliminación de un enlace, desvincula un nombre de los datos reales. Los datos todavía estarán accesibles mientras quede algún enlace. Cuando se elimina el último enlace duro, el espacio que ocupaban los datos se considera disponible.

Soft link: Indica un acceso a un directorio o fichero que se encuentra en un lugar distinto dentro de la estructura de directorios. Una modificación realizada utilizando este enlace se reflejará en el original; pero, por el contrario, si se elimina el enlace, no se eliminará el auténtico. Per a fer aquestes còpies utilitzarem `tar` i `rsync`.

Partició per emmagatzemar les còpies de seguretat

1. Crear (com a root) un nou directori on muntarem l'espai de disc:

   ```
   mkdir /backup
   ```

2. Crear una nova partició a l'espai lliure del disc amb `btrfs` i `gdisk`:

   a) Si la zona està buida (sense particions prèvies): `mkfs.btrfs /dev/usb4`

   ```
   gdisk /dev/usb
   # dins gdisk:
   #  n        nova partici
   #  [enter]     selecci  de  l ID  per defecte
   #  +20G     mida de 20 GiB
   #  t        elegir tipus, per exemple 8300 (Linux filesystem)
   #  w        escriure canvis al disc
   ```

3. Muntar la nova partició:

   ```
   mount /dev/sdb4 /backup
   ```

4. Concedir només accés a root:

   ```
   chmod go-rx /backup
   ```

5. Per motius de seguretat, muntar en mode només lectura i remuntar en escriptura quan fem el backup:

   ```
   mount -o remount,ro /dev/sdb4 /backup
   # ... fer el backup ...
   mount -o remount,rw /dev/sdb4 /backup
   ```

6. Modificar `/etc/fstab` perquè la partició es monti automàticament en arrencar:

   ```
   # /etc/fstab
   /dev/sdb4   /backup   btrfs   ro   0 2
   ```

Còpies amb `tar`

```
tar -cf /backup/backup-etc-`date +%Y%m%d%H%M%S`.tar /etc
```

Per activar compressió `-z` (gzip):

```
tar -czf /backup/backup-etc-`date +%Y%m%d%H%M%S`.tar.gz /etc
```

Excloure fitxers Si volem excloure certs camins, fem un fitxer `excludes.txt` amb cada patron en una línia, i:

```
tar -czf /backup/backup-etc-`date +%Y%m%d%H%M%S`.tar.gz --exclude-from=excludes.txt
/etc
```

Un mecanisme de signatura digital, com són els hashes `sha512sum`, permet verificar la integritat d'un fitxer i comprovar-la:

```
# sha512sum <file> > <file>.asc
# sha512sum -c <file>.asc <file>
```

Còpies Incrementals Les còpies incrementals fan backup només dels fitxers modificats. Per provar-ho, podem canviar la data de modificació d'alguns fitxers amb `touch`.

`tar` té l'opció `-newer` que només inclou els fitxers modificats després d'una data concreta.

- Especificant data manualment:

  ```
  tar --newer='2007-11-28 12:10' -czf incr.tar.gz /directori
  ```

- Agafant la data d'un fitxer de referència:

  ```
  tar --newer=<file> -czf incr.tar.gz /directori
  ```

Recorda que el directori de backup podria estar en mode Read-Only.

Restauració de Còpies Primer restaurar la còpia total, i després, en ordre, les incrementals. S'extrau l'arxiu al directori de destí:

```
tar xvf total.tar -C /dest
tar xvf incr1.tar -C /dest
tar xvf incr2.tar -C /dest
```

Restauració de Fragments `tar` permet extreure fitxers o rutes concretes:

```
# tar xvf file.tar <filetoextract>
# tar xvf file.tar [path/to/dir | path/to/file]
```

Per indicar un directori de destí diferent:

```
tar xvf file.tar -C /target/dir path/to/file
```

Rsync `rsync` permet copiar un directori (o un conjunt de fitxers) a un altre directori via xarxa, utilitzant un algorisme de checksum per transmetre només les diferències. Comprimeix les dades en vol per accelerar la transferència i preserva enllaços, dispositius, permisos, grups i propietaris. També suporta llistats d'exclusió i connexions SSH segures.

Backups a Traves de Xarxa Exemple de còpia remota amb `rsync`:

```
rsync -avz /root -e ssh root@localhost:/backup/backup-rsync/
```

Cal tenir el compte root actiu amb una contrasenya vàlida i SSH instal·lat.

Còpies Incrementals Inverses El directori on tenim el *mirror* queda exactament igual que el directori d'origen, sense deixar rastre dels canvis previs. Per solucionar-ho, `rsync` ofereix les opcions `-backup` i `-backup-dir`. Els backups generats amb aquestes opcions s'anomenen *inversos* perquè la còpia *completa* correspon a l'última data de backup i les incremental es troben en subdirectoris datats.

Listing 5: Exemple d'script per a còpies incrementals inverses

```
#!/bin/bash

SOURCE_DIR=/origen/path
DEST_DIR=/backup/path

# Fitxer amb patrons a excloure
EXCLUDES=/etc/backup-excludes.txt

# Nom o IP de la m quina de backup
BSERVER=backup.example.com

# Data per al directori incremental (YYYYMMDDHHMMSS)
BACKUP_DATE=$(date +%Y%m%d%H%M%S)

# Opcions per a rsync
OPTS="--ignore-errors --delete-excluded \
      --exclude-from=\$EXCLUDES \
      --delete --backup \
      --backup-dir=\$DEST_DIR/\$BACKUP_DATE \
      -av"

# Execuci  de la c pia
rsync \$OPTS \$SOURCE_DIR root@\${BSERVER}:\$DEST_DIR/complet
```